ADRC_v2

July 29, 2026

1 Active Disturbance Rejection Control

[2]:
```
import numpy as np
from sympy import *
from sympy import Matrix, symbols, init_printing
from sympy import Function, laplace_correspondence, diff, solve,symbols
import matplotlib.pyplot as plt
from control.matlab import *
from IPython.display import display, Latex
init_printing(use_latex='mathjax')
```

[3]:
```
# ============================================================
# Helper functions: sympy <-> control transfer function
# ============================================================

def tf2sym(tf, symb=None):
    """Convert a control.TransferFunction into a sympy expression."""
    s = symbols(symb) if symb is not None else symbols('s')
    numTF = tf.num[0][0]
    denTF = tf.den[0][0]
    num_symb = sum(numTF[i] * s**(len(numTF) - 1 - i) for i in␣
 ↪range(len(numTF)))
    den_symb = sum(denTF[i] * s**(len(denTF) - 1 - i) for i in␣
 ↪range(len(denTF)))
    return (num_symb / den_symb).simplify()


def sym2tf(expr, symb=None, Ts=None):
    """Convert a sympy expression into a control.TransferFunction."""
    s = symbols(symb) if symb is not None else symbols('s')
    num, den = fraction(expr)
    num_poly = Poly(num, s)
    den_poly = Poly(den, s)

    num_coeff = [float(i) for i in num_poly.as_poly().all_coeffs()]
    den_coeff = [float(i) for i in den_poly.as_poly().all_coeffs()]
```

```
    if Ts is not None:
        return tf(num_coeff, den_coeff, Ts)
    return tf(num_coeff, den_coeff)
```

1.1 PLANT

2 Motor transfer function

The motor transfer function is given by

$$G(s) = \frac{\theta(s)}{U(s)} = \frac{k_\omega}{\tau s + 1} \cdot \frac{k_\theta}{s} = \frac{k}{s(\tau s + 1)}.$$

where k_ω and k_θ are gains for unit conversion, for example, ω in rev/s and θ in degrees.

A state-space representation with

$$x_p(t) = \begin{bmatrix} \theta(t) \\ \omega(t) \end{bmatrix}$$

results in

$$\dot{x}_p(t) = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{1}{\tau} \end{bmatrix} x_p(t) + \begin{bmatrix} 0 \\ \frac{k}{\tau} \end{bmatrix} u(t),$$

and

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x_p(t).$$

After discretization we obtain

$$x_p(k+1) = A_p x_p(k) + B_p u(k),$$

$$y(k) = C_p x_p(k).$$

[4]:
```
# ============================================================
# Design specifications
# ============================================================
M_p = 25          # overshoot (%)
e_v = 0           # velocity error
e_a = 0.001       # acceleration error
K_a = 1 / e_a     # acceleration error constant
m_f = 40          # phase margin target (deg)
m_g = 10          # gain margin target (dB)
T_s = 0.005       # sampling period (s)
```

```
K_theta = 360
```

[5]:
```
# ============================================================
# Plant obtained with system identification
# ============================================================
num  = [2201]
den  = [1, 45.08]
Gs   = tf(num, den)

Tau  = 1 / den[1]
K_dc = num[0] * (1 / den[1])

ts   = 4.5 * Tau

# Symbolic form
s, T, K, z, Ki = symbols(['s', 'T', 'K', 'z', 'K_i'])
Gs_symb = K /(s*(T*s + 1))
print(Gs_symb)
Ap = np.array([
    [0,       1],
    [0, -1/Tau]
])

Bp = np.array([
    [0],
    [K_dc*K_theta / Tau]
])

Cp = np.array([
    [1, 0]
])

Dp = np.array([
    [0]
])
Gs_ss = ss(Ap,Bp,Cp,Dp)
display(Gs_ss)

Gs_symb = Gs_symb.subs({K:K_dc*K_theta,T:Tau})
print('Continuous-time symbolic:')
display(Gs_symb)

display(Latex(r'$T$'));       display(Tau)
display(Latex(r'$t_s:$'));    display(ts)
display(Latex(r'$K_{dc}:$')); display(K_dc*K_theta)
```

```
K/(s*(T*s + 1))
```

<StateSpace sys[1]: ['u[0]'] -> ['y[0]']>

$$\left[\begin{array}{cc|c} 0 & 1 & 0 \\ 0 & -45.1 & 7.92{\cdot}10^{5} \\ \hline 1 & 0 & 0 \end{array}\right]$$

Continuous-time symbolic:

$$\frac{17576.7524401065}{s\left(0.0221827861579414s + 1\right)}$$

T

0.0221827861579414

t_s :

0.0998225377107365

K_{dc} :

17576.7524401065

[6]:
```
# ============================================================
# Continuous- and discrete-time plant
# ============================================================

Gz_ss  = c2d(Gs_ss, T_s)
#Gz_ss = c2d(tf2ss(sym2tf(Gs_symb)),T_s)
print('Continuous time:')
display(Gs_ss)
print('Discrete time (ZOH):')
display(Gz_ss)

print('Discrete time  State Space (ZOH):')
display(Gz_ss)

Ap_z = Gz_ss.A
Bp_z = Gz_ss.B
Cp_z = Gz_ss.C
Dp_z = Gz_ss.D
```

Continuous time:

<StateSpace sys[1]: ['u[0]'] -> ['y[0]']>

$$\left[\begin{array}{cc|c} 0 & 1 & 0 \\ 0 & -45.1 & 7.92{\cdot}10^{5} \\ \hline 1 & 0 & 0 \end{array}\right]$$

Discrete time (ZOH):

<StateSpace sys[1]$sampled: ['u[0]'] -> ['y[0]'], dt=0.005>

$$\left[\begin{array}{ll|l} 1 & 0.00448 & 9.2 \\ 0 & 0.798 & 3.55{\cdot}10^{3} \\ \hline 1 & 0 & 0 \end{array}\right]$$

```
Discrete time  State Space (ZOH):
```

<StateSpace sys[1]$sampled: ['u[0]'] -> ['y[0]'], dt=0.005>

$$\left[\begin{array}{ll|l} 1 & 0.00448 & 9.2 \\ 0 & 0.798 & 3.55{\cdot}10^{3} \\ \hline 1 & 0 & 0 \end{array}\right]$$

[7]:
```
# ============================================================
# Step response of plant (continuous vs. discrete)
# ============================================================
y,  t  = step(Gs_ss)
yz, tz = step(Gz_ss)

# Settling time (2% band) on the continuous response


print(f'steady state: {y[-1]}')

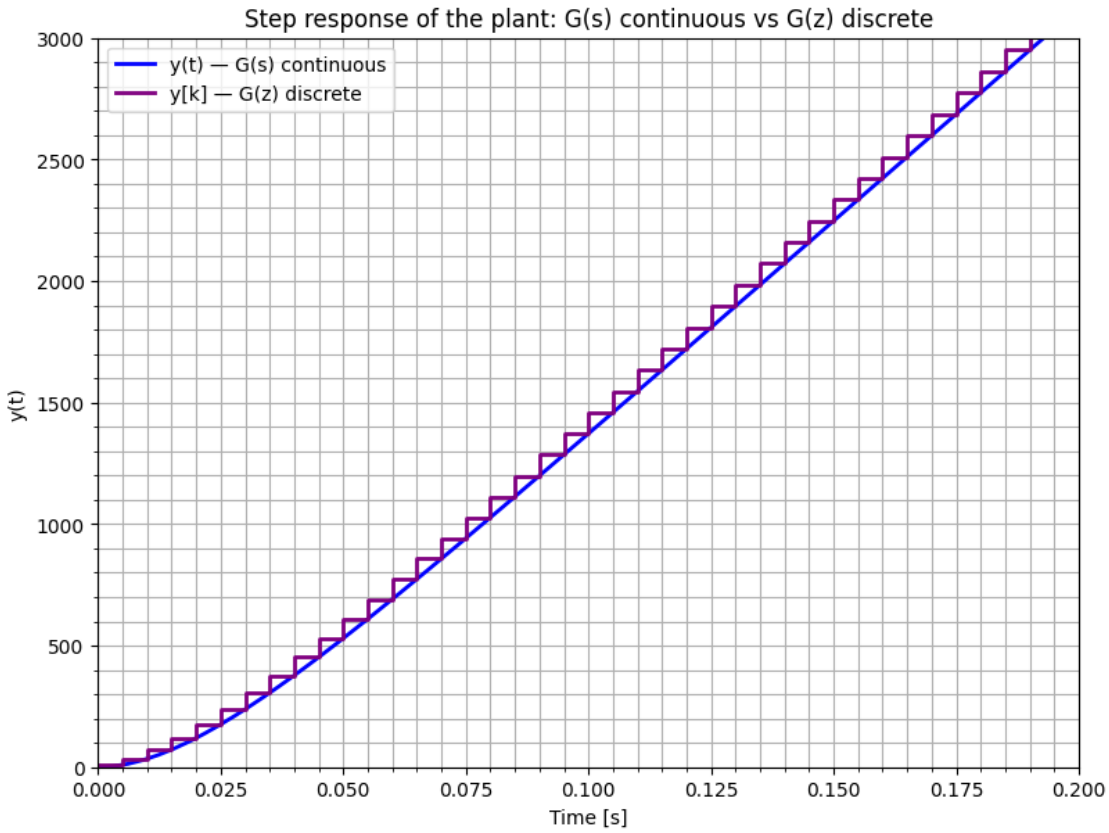
fig, ax = plt.subplots(1, 1, figsize=(8, 6), layout='constrained')

ax.plot(t,  y,  color='blue',   linewidth=2, linestyle='-', label='y(t) - G(s)␣
 ↪continuous')
ax.step(tz, yz, color='purple', linewidth=2, linestyle='-', label='y[k] - G(z)␣
 ↪discrete')

ax.set_title('Step response of the plant: G(s) continuous vs G(z) discrete')
ax.set_xlabel('Time [s]')
ax.set_ylabel('y(t)')
ax.set_xlim(0, 0.2)
ax.set_ylim(0, 3000)
ax.minorticks_on()
ax.grid(visible=True, which='both')
ax.legend()
```

```
steady state: 13076.813882817956
```

[7]:
```
<matplotlib.legend.Legend at 0x78324a9771a0>
```



3 Two Integral

The internal model is described by

$$x_\xi(k+1) = A_\xi x_\xi(k) + B_\xi\, \xi^{(m)}(k),$$

$$\xi(k) = C_\xi x_\xi(k).$$

[8]:
```
#State Space disturbance system
sys = tf([1],[1,0])
Ad = np.block([[0,1],[0,0]])
Bd = np.block([[1],[0]])
Cd = np.block([[1,0]])
Dd = np.block([[0]])
sys_d_z = c2d(ss(Ad,Bd,Cd,Dd),T_s)
display(Latex(r"State Space of Disturbance System:"))
display(sys_d_z)
Ad_z = sys_d_z.A
```

```
Bd_z = sys_d_z.B
Cd_z = sys_d_z.C
Dd_z = sys_d_z.D
```

State Space of Disturbance System:

<StateSpace sys[4]$sampled: ['u[0]'] -> ['y[0]'], dt=0.005>

$$\left[\begin{array}{ll|l} 1 & 0.005 & 0.005 \\ 0 & 1 & 0 \\ \hline 1 & 0 & 0 \end{array}\right]$$

3.1 Augmented system

3.1.1 Extended System

The augmented discrete-time system is

$$\begin{bmatrix} x_p(k+1) \\ \xi(k+1) \end{bmatrix} = \begin{bmatrix} A_p & B_p C_\xi \\ 0_{(m,n)} & A_\xi \end{bmatrix} \begin{bmatrix} x_p(k) \\ \xi(k) \end{bmatrix} + \begin{bmatrix} B_p \\ 0_{(m,1)} \end{bmatrix} u(k) + \begin{bmatrix} 0_{(n,1)} \\ B_\xi \end{bmatrix} \xi^{(m)}(k).$$

The output equation is

$$y(k) = \begin{bmatrix} C_p & 0_{(1,m)} \end{bmatrix} \begin{bmatrix} x_p(k) \\ \xi(k) \end{bmatrix}.$$

In compact form,

$$x_{au}(k+1) = A_{au} x_{au}(k) + B_{au} u(k) + B_{\xi,au}\, \xi^{(m)}(k),$$

$$y(k) = C_{au} x_{au}(k).$$

where the augmented state vector is

$$x_{au}(k) = \begin{bmatrix} x_p(k) \\ \xi(k) \end{bmatrix}.$$

[9]:
```
n=Ap.shape[0]
m=Ad.shape[0]

Aau = np.block([[Ap_z,Bp_z@Cd_z],
                [np.zeros((m,n)),Ad_z]])
Bau = np.block([[Bp_z],
                [np.zeros((m,1))]])
Cau = np.block([Cp,np.zeros((1,m))])
Dau = np.block([np.zeros((1,1))])
sys_au_z = ss(Aau,Bau,Cau,Dau,T_s)
```

```
display(Latex(r"State Space of Augmented System:"))
display(sys_au_z)
```

State Space of Augmented System:

<StateSpace sys[6]: ['u[0]'] -> ['y[0]'], dt=0.005>

$$\left[\begin{array}{cccc|c} 1 & 0.00448 & 9.2 & 0 & 9.2 \\ 0 & 0.798 & 3.55{\cdot}10^{3} & 0 & 3.55{\cdot}10^{3} \\ 0 & 0 & 1 & 0.005 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ \hline 1 & 0 & 0 & 0 & 0 \end{array}\right]$$

3.2 Observer System

The discrete-time observer is

$$\hat{x}_{au}(k+1) = (A_{au} - LC_{au})\,\hat{x}_{au}(k) + B_{au}u(k) + Le_y(k).$$

where

$$e_y(k) = C_{au}x_{au}(k).$$

3.2.1 Estimation Error Dynamics

Defining the estimation error as

$$\bar{e}(k) = x_{au}(k) - \hat{x}_{au}(k),$$

its dynamics are

$$\bar{e}(k+1) = (A_{au} - LC_{au})\,\bar{e}(k) + B_{\xi,au}\,\xi^{(m)}(k).$$

3.2.2 State Feedback Control Law

The control law is

$$u(k) = -k_1\hat{x}_{au,1}(k) - k_2\hat{x}_{au,2}(k) - \cdots - k_{n+m}\hat{x}_{au,n+m}(k),$$

or, in compact form,

$$u(k) = -K\hat{x}_{au}(k).$$

Since

$$\hat{x}_{au}(k) = \begin{bmatrix} \hat{x}_p(k) \\ \hat{\xi}(k) \end{bmatrix},$$

the control law can also be written as

$$u(k) = -K_p \hat{x}_p(k) - 1 * \hat{\xi}(k).$$

3.3 Closed-Loop Augmented System

The closed-loop dynamics are

$$\begin{bmatrix} x_{au}(k+1) \\ \bar{e}(k+1) \end{bmatrix} = \begin{bmatrix} A_{au} - B_{au}K & B_{au}K \\ 0 & A_{au} - LC_{au} \end{bmatrix} \begin{bmatrix} x_{au}(k) \\ \bar{e}(k) \end{bmatrix} \cdot u(k)$$

[10]:
```
pols = np.array([
    -30,
    -20,
])
# Discrete-time poles
poles_d = np.exp(pols * T_s)
K_1= acker(Ap_z,Bp_z,poles_d).reshape(1,n)
K = np.block([K_1,1,0]).reshape(1,n+m)
display(Latex(r"State Feedback Gain K:"))
display(K_1)
display(K)

pols_L = np.array([
-280,-300,-320,-340
])
# Convert to discrete time
poles_L_d = np.exp(pols_L * T_s)
L = acker(Aau.T,Cau.T, poles_L_d.T).reshape(n+m,1)
display(Latex(r"Observer Gain L:"))
display(L)

# Closed-loop poles
eig_cl = np.linalg.eigvals(Ap_z - Bp_z @ K_1)

# Observer poles
eig_obs = np.linalg.eigvals(Aau - L @ Cau)
print("Desired controller poles :", np.sort_complex(poles_d))
print("Obtained controller poles:", np.sort_complex(eig_cl))

print("Desired observer poles :", np.sort_complex(poles_L_d))
print("Obtained observer poles:", np.sort_complex(eig_obs))
```

State Feedback Gain K:

```
array([[7.47404898e-04, 7.26661999e-06]])
```

```
array([[7.47404898e-04, 7.26661999e-06, 1.00000000e+00, 0.00000000e+00]])
```

Observer Gain L:

```
array([[2.94388971e+00],
       [4.93675336e+02],
       [9.84279037e-02],
       [4.30545007e+00]])
```

```
Desired controller poles : [0.86070798+0.j 0.90483742+0.j]
Obtained controller poles: [0.86070798+0.j 0.90483742+0.j]
Desired observer poles : [0.18268352+0.j 0.20189652+0.j 0.22313016+0.j
0.24659696+0.j]
Obtained observer poles: [0.18268352+0.j 0.20189652+0.j 0.22313016+0.j
0.24659696+0.j]
```

[11]:

```
# ============================================================
# Time vectors
# ============================================================

# Continuous time
t = np.linspace(0, 1.0, 1000)

# Discrete time
k = np.arange(0, int(1 / T_s) + 1)

# ============================================================
# CONTROLLER DYNAMICS
# ============================================================

# Continuous dynamics
responses_controller = np.array([
    np.exp(p * t)
    for p in pols
])

# Discrete dynamics
responses_controller_d = np.array([
    p ** k
    for p in poles_d
])


# ============================================================
# Plot Controller Dynamics
# ============================================================

fig, ax = plt.subplots(
    figsize=(8, 6),
    layout='constrained'
```

```
)

# Continuous poles
for i, p in enumerate(pols):

    ax.plot(
        t,
        responses_controller[i],
        linewidth=2,
        linestyle='-',
        label=fr'Continuous: $s={p}$'
    )

# Discrete poles
for i, p in enumerate(poles_d):

    ax.step(
        k * T_s,
        responses_controller_d[i],
        where='post',
        linewidth=2,
        linestyle='--',
        label=fr'Discrete: $z={p:.4f}$'
    )

ax.set_title(
    'Controller Pole Dynamics'
)

ax.set_xlabel(
    'Time [s]'
)

ax.set_ylabel(
    r'Modal response'
)

ax.set_xlim(
    0,
    0.20
)

ax.set_ylim(
    0,
    1.05
)
```

```
ax.minorticks_on()

ax.grid(
    visible=True,
    which='both'
)

ax.legend()

plt.show()


# ============================================================
# OBSERVER DYNAMICS
# ============================================================

# Continuous dynamics
responses_observer = np.array([
    np.exp(p * t)
    for p in pols_L
])

# Discrete dynamics
responses_observer_d = np.array([
    p ** k
    for p in poles_L_d
])


# ============================================================
# Plot Observer Dynamics
# ============================================================

fig, ax = plt.subplots(
    figsize=(8, 6),
    layout='constrained'
)

# Continuous poles
for i, p in enumerate(pols_L):

    ax.plot(
        t,
        responses_observer[i],
        linewidth=2,
        linestyle='-',
        label=fr'Continuous: $s={p}$'
```

```
    )

# Discrete poles
for i, p in enumerate(poles_L_d):

    ax.step(
        k * T_s,
        responses_observer_d[i],
        where='post',
        linewidth=2,
        linestyle='--',
        label=fr'Discrete: $z={p:.4f}$'
    )

ax.set_title(
    'Observer Pole Dynamics'
)

ax.set_xlabel(
    'Time [s]'
)

ax.set_ylabel(
    r'Modal response'
)

ax.set_xlim(
    0,
    0.1
)

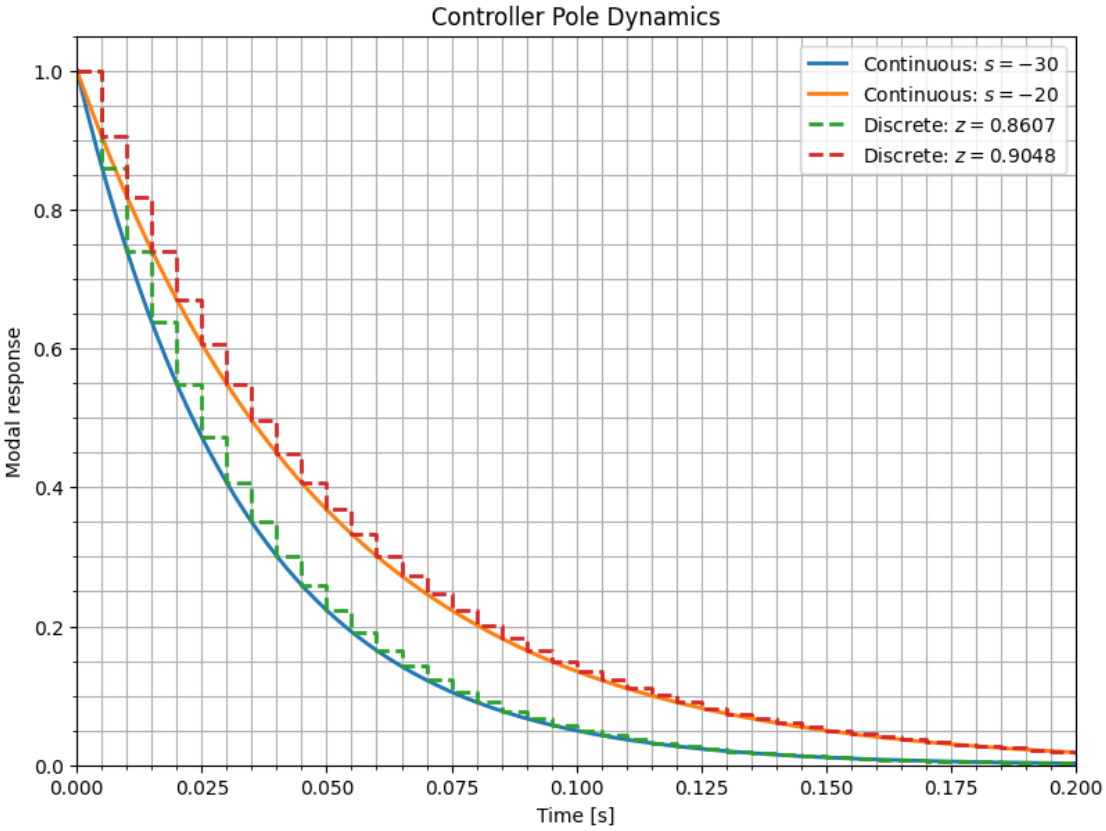
ax.set_ylim(
    0,
    1.05
)

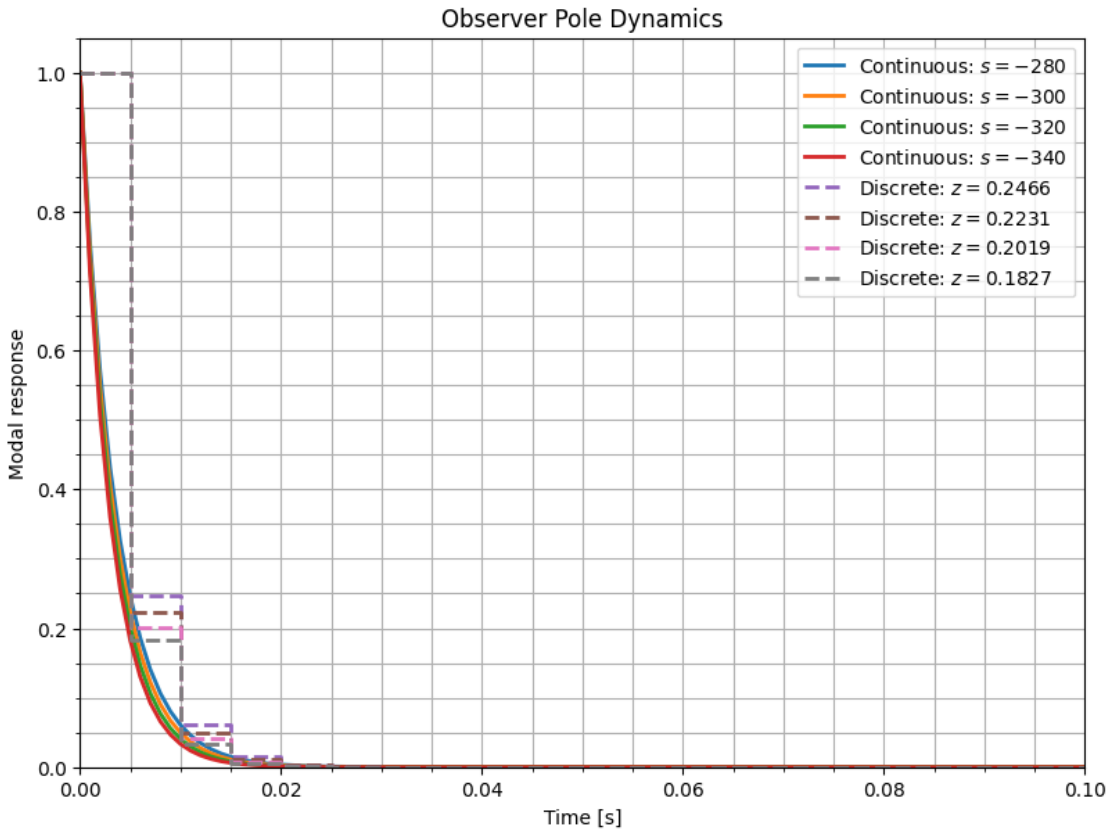
ax.minorticks_on()

ax.grid(
    visible=True,
    which='both'
)

ax.legend()

plt.show()
```


Controller Pole Dynamics
Continuous: $s = -30$
Continuous: $s = -20$
Discrete: $z = 0.8607$
Discrete: $z = 0.9048$
Modal response
Time [s]
0.0
0.2
0.4
0.6
0.8
1.0
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200



[12]:
```
#######################################
#Observer
#######################################
Ao = (Aau - L@Cau)
print(Ao)
Bo = np.block([L,Bau])
Co = np.ones(4)

Do = np.zeros_like((4,2))
sys_o = ss(Ao,Bo,Co,Do,T_s)
print(sys_o)

print(L)
```

```
[[-1.94388971e+00  4.47655554e-03  9.20045365e+00  0.00000000e+00]
 [-4.93675336e+02  7.98196876e-01  3.54704355e+03  0.00000000e+00]
 [-9.84279037e-02  0.00000000e+00  1.00000000e+00  5.00000000e-03]
 [-4.30545007e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
<StateSpace>: sys[7]
Inputs (2): ['u[0]', 'u[1]']
Outputs (1): ['y[0]']
```

```
States (4): ['x[0]', 'x[1]', 'x[2]', 'x[3]']
dt = 0.005

A = [[-1.94388971e+00  4.47655554e-03  9.20045365e+00  0.00000000e+00]
     [-4.93675336e+02  7.98196876e-01  3.54704355e+03  0.00000000e+00]
     [-9.84279037e-02  0.00000000e+00  1.00000000e+00  5.00000000e-03]
     [-4.30545007e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]

B = [[2.94388971e+00 9.20045365e+00]
     [4.93675336e+02 3.54704355e+03]
     [9.84279037e-02 0.00000000e+00]
     [4.30545007e+00 0.00000000e+00]]

C = [[1. 1. 1. 1.]]

D = [[0. 0.]]
[[2.94388971e+00]
 [4.93675336e+02]
 [9.84279037e-02]
 [4.30545007e+00]]
```

3.4 Closed-Loop System for simulation

The discrete-time closed-loop system is

$$\begin{bmatrix} x_p(k+1) \\ \hat{x}_{au}(k+1) \end{bmatrix} = \begin{bmatrix} A_p & -B_pK \\ LC_p & A_{au} - LC_{au} - B_{au}K \end{bmatrix} \begin{bmatrix} x_p(k) \\ \hat{x}_{au}(k) \end{bmatrix} + \begin{bmatrix} 0_{(n,1)} & B_p \\ -L & 0_{(n+m,1)} \end{bmatrix} \begin{bmatrix} r(k) \\ \xi(k) \end{bmatrix}.$$

[13]:
```
## Estended state observeri with the plant system:

Aau_e = np.block([[Ap_z,-Bp_z@K],
                  [L@Cp_z,Aau - L@Cau - Bau@K]])
Bau_e = np.block([[np.zeros((n,1)),Bp_z],
                  [-L,np.zeros((n+m,1))]])

Cau_e = np.block([Cau,np.zeros((1,2))])
Dau_e = np.block([np.zeros((1,2))])

print(Aau_e.shape)
print(Bau_e.shape)
print(Cau_e.shape)
print(Dau_e.shape)
sys_au_e = ss(Aau_e,Bau_e,Cau_e,Dau_e,T_s)
display((sys_au_e))
display(Latex(r"State Space of Extended System:"))
```

```
(6, 6)
(6, 2)
```

```
(1, 6)
(1, 2)
```

<StateSpace sys[8]: ['u[0]', 'u[1]'] -> ['y[0]'], dt=0.005>

$$\left[\begin{array}{cccccc|cc}
1 & 0.00448 & -0.00688 & -6.69\cdot10^{-5} & -9.2 & 0 & 0 & 9.2 \\
0 & 0.798 & -2.65 & -0.0258 & -3.55\cdot10^{3} & 0 & 0 & 3.55\cdot10^{3} \\
2.94 & 0 & -1.95 & 0.00441 & 0 & 0 & -2.94 & 0 \\
494 & 0 & -496 & 0.772 & 0 & 0 & -494 & 0 \\
0.0984 & 0 & -0.0984 & 0 & 1 & 0.005 & -0.0984 & 0 \\
4.31 & 0 & -4.31 & 0 & 0 & 1 & -4.31 & 0 \\
\hline
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{array}\right]$$

State Space of Extended System:

[14]:
```
t=np.arange(0,4,T_s)
r=np.ones_like(t)*t*360/4
xi = (t >= 0.3) * 0.001  # sin perturbacion en la simulacion (xi = 0)
u = np.column_stack([r, xi])
xp0=np.zeros(n)
xp0_es= np.zeros(n)
xd0_es= np.zeros(m)



# Closed-loop outputs

x0=  np.block([xp0,xp0_es,xd0_es])
[Y,T,X] = lsim(sys_au_e, U=u, T=t, X0=x0)
print(X[-1,0])
print(X.shape)
Y1 = X[:,0]

xhat_au = X[:, n:]
u_ctrl = -(K @ xhat_au.T).ravel()   # el control que de verdad se aplica es u =␣
 ↪-K x_hat
```

```
359.5499999999987
(800, 6)
```

[15]:
```
fig, axs = plt.subplots(3, 2, figsize=(12,9), layout='constrained')

# Salida vs referencia
axs[0,0].step(T, Y, color='C3', linewidth=2, label=r'$\theta$ (output)')
axs[0,0].step(T, r, color='C0', linewidth=2, label='r (reference)')
axs[0,0].set_title(r'Output $\theta$ vs reference')
axs[0,0].set_ylabel('angle [°]')
axs[0,0].set_ylim(0,45)
# Error de posicion: estimado por el observador vs error real
```

```
axs[0,1].step(T, X[:,2],   color='C3', linewidth=2, label=r'$\hat{x}_1$␣
 ↪(estimated)')
axs[0,1].step(T, X[:,0]-r, color='C0', linewidth=2, label=r'$\theta-r$ (real)')
axs[0,1].set_title('Position error: estimated vs real')
axs[0,1].set_ylabel('error [°]')


# Derivada del error estimada
axs[1,0].step(T, X[:,3], color='C1', linewidth=2, label=r'$\hat{x}_2$')
axs[1,0].set_title('Estimated error derivative')

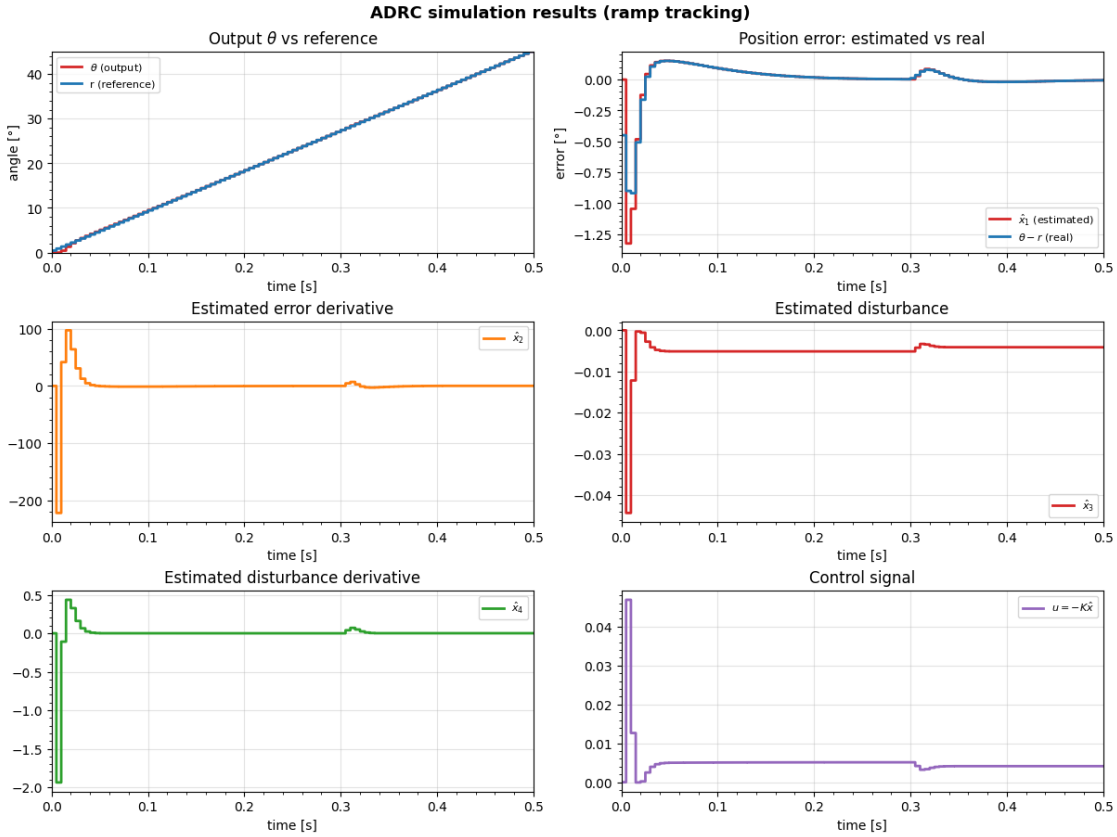
# Perturbacion estimada
axs[1,1].step(T, X[:,4], color='C3', linewidth=2, label=r'$\hat{x}_3$')
axs[1,1].set_title('Estimated disturbance')

# Derivada de la perturbacion estimada
axs[2,0].step(T, X[:,5], color='C2', linewidth=2, label=r'$\hat{x}_4$')
axs[2,0].set_title('Estimated disturbance derivative')

# Senal de control
axs[2,1].step(T, u_ctrl, color='C4', linewidth=2, label=r'$u=-K\hat{x}$')
axs[2,1].set_title('Control signal')

for ax in axs.flat:
    ax.minorticks_on(); ax.legend(fontsize=8); ax.grid(visible=True,␣
 ↪which='major', alpha=0.35)
    ax.set_xlabel('time [s]')
    ax.set_xlim(0,0.5)
fig.suptitle('ADRC simulation results (ramp tracking)', fontsize=13,␣
 ↪fontweight='bold')
```

[15]: Text(0.5, 0.98, 'ADRC simulation results (ramp tracking)')



4 Resultados experimentales

Se cargan los cuatro conjuntos de datos adquiridos del banco, con las columnas `ep`, `theta`, `vel`, `r`, `xhat0`–`xhat3` (donde `xhat0`=$\hat{x}_1$ error de posición, `xhat1`=$\hat{x}_2$, `xhat2`=$\hat{x}_3$ perturbación, `xhat3`=$\hat{x}_4$).

Como la referencia es trapezoidal periódica, se analiza **una sola subida** (0°→360°) ubicada aproximadamente en la mitad de cada registro.

[16]:
```
import pandas as pd

# Carga y alineacion a una subida cercana a la mitad de cada registro
def cargar_subida(fname, dur=4.2):
    df = pd.read_csv(fname)
    p  = df.pivot_table(index='x', columns='variablename', values='y').
 ↪interpolate().dropna()
    p.index = p.index - p.index.values[0]
    t = p.index.values; r = p['r'].values; dr = np.gradient(r, t)
    mid = (t[-1]-t[0])/2
    subidas = [t[i] for i in range(1,len(t)-1) if r[i]<20 and dr[i]>40 and␣
 ↪r[i-1]<=r[i]+1]
```

```
    subidas = [s for j,s in enumerate(subidas) if j==0 or s-subidas[j-1]>5]
    t0 = min(subidas, key=lambda s: abs(s-mid))
    q = p[(p.index>=t0-0.02) & (p.index<t0+dur)].copy(); q.index = q.index - q.
 ↪index.values[0]
    return q

R = {
    'real_con'  : cargar_subida('logs/ep_real_con.csv'),
    'real_sin'  : cargar_subida('logs/ep_real_sin.csv'),
    'estim_con' : cargar_subida('logs/ep_stimation_con.csv'),
    'estim_sin' : cargar_subida('logs/ep_stimation_sin.csv'),
}
LB = {'real_con':'Using real error, with disturbance','real_sin':'Using real␣
 ↪error, without disturbance.',
      'estim_con':' Using estimated error, with disturbance.','estim_sin':
 ↪'Using estimated error, without disturbance.'}
CO = {'real_con':'C3','real_sin':'C0','estim_con':'C1','estim_sin':'C2'}
print('Subidas cargadas:', {k: len(v) for k,v in R.items()})
```

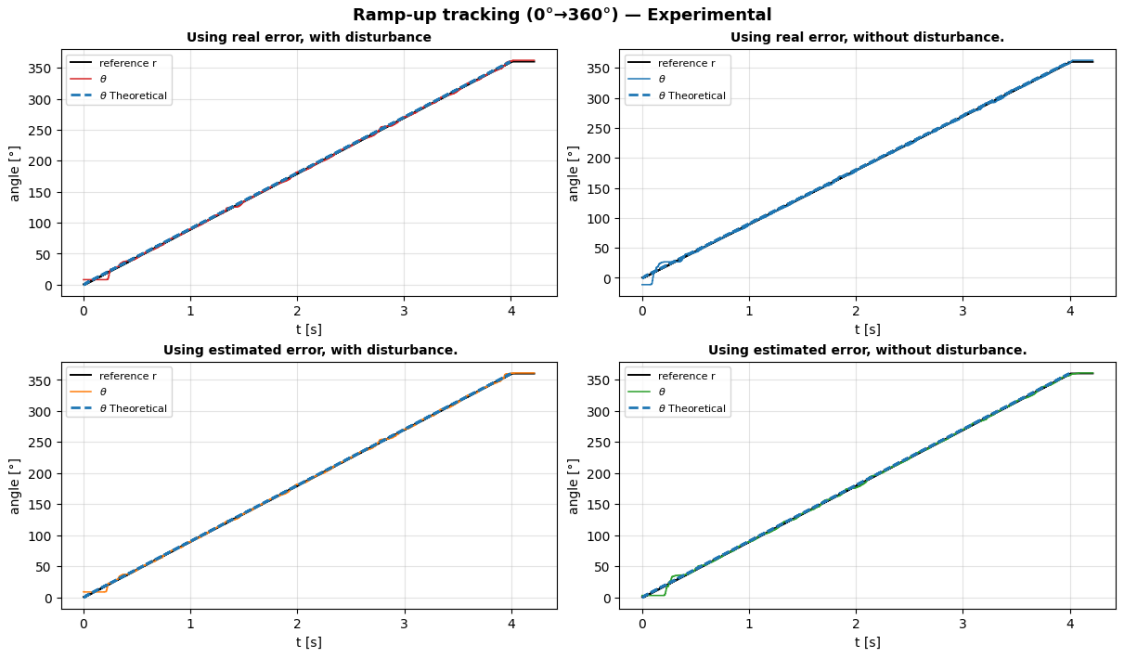
```
Subidas cargadas: {'real_con': 845, 'real_sin': 844, 'estim_con': 844,
'estim_sin': 844}
```

[17]:
```
# Simulacion teorica del lazo cerrado (mismas matrices del diseno) para la␣
 ↪rampa de subida
Ao_ = np.asarray(Aau) - np.asarray(L) @ np.asarray(Cau)
Lo_ = np.asarray(L).ravel(); Bo_ = np.asarray(Bau).ravel(); K1_ = np.
 ↪asarray(K_1).ravel()
Apz_ = np.asarray(Ap_z); Bpz_ = np.asarray(Bp_z)
def _sat(x): return max(-1.0, min(1.0, x))
def sim_teorica(N=800):
    x = np.zeros((2,1)); xh = np.zeros(4)
    o = {k: np.zeros(N) for k in␣
 ↪['t','r','theta','ep','u','xhat0','xhat1','xhat2','xhat3','vel']}
    for k in range(N):
        t = k*T_s; r = 90.0*t; p = x[0,0]; ep = p - r
        u = _sat(-(K1_[0]*xh[0] + K1_[1]*xh[1] + xh[2]))
        o['t'][k]=t; o['r'][k]=r; o['theta'][k]=p; o['ep'][k]=ep; o['u'][k]=u;␣
 ↪o['vel'][k]=x[1,0]
        o['xhat0'][k]=xh[0]; o['xhat1'][k]=xh[1]; o['xhat2'][k]=xh[2];␣
 ↪o['xhat3'][k]=xh[3]
        x = Apz_ @ x + Bpz_ * u; xh = Ao_ @ xh + Lo_ * ep + Bo_ * u
    return o
TEO = sim_teorica()
print('Simulacion teorica lista (rampa 90 deg/s).')
```

```
Simulacion teorica lista (rampa 90 deg/s).
```

4.0.1 Seguimiento de la rampa de subida (0°→360°) — Experimental

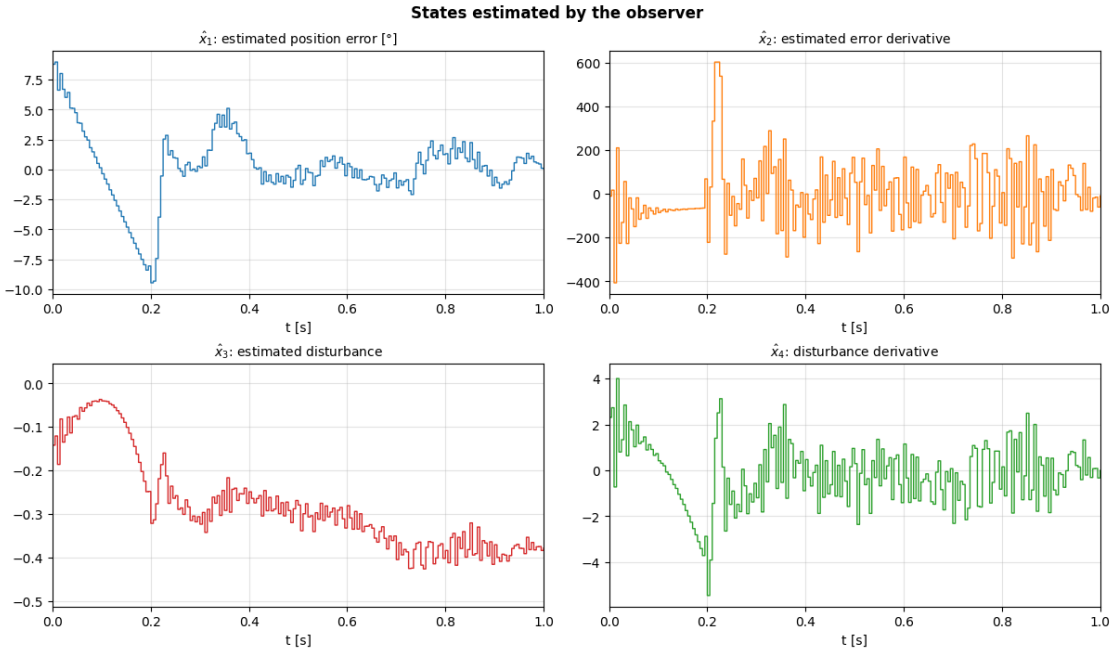
[18]:
```
fig, axs = plt.subplots(2, 2, figsize=(12,7), layout='constrained')
for a, k in zip(axs.flat, R):
    p = R[k]
    a.step(p.index, p['r'], 'k', lw=1.4, label='reference r')
    a.step(p.index, p['theta'], CO[k], lw=1.1, label=r'$\theta$')
    a.step(T,Y, lw=2, linestyle='--',label=r'$\theta$ Theoretical')
    a.set_title(LB[k], fontsize=10, fontweight='bold'); a.set_xlabel('t [s]');␣
 ↪a.set_ylabel('angle [°]')
    a.grid(alpha=0.35); a.legend(fontsize=8)
fig.suptitle('Ramp-up tracking (0°→360°) - Experimental', fontsize=13,␣
 ↪fontweight='bold')
plt.show()
```



4.0.2 Estados estimados por el observador en la subida (control con estimación, con perturbación)

[19]:
```
p = R['estim_con']
fig, axs = plt.subplots(2, 2, figsize=(12,7), layout='constrained')
est = [('xhat0', r'$\hat{x}_1$: estimated position error [°]', 'C0'),
       ('xhat1', r'$\hat{x}_2$: estimated error derivative',     'C1'),
       ('xhat2', r'$\hat{x}_3$: estimated disturbance',          'C3'),
       ('xhat3', r'$\hat{x}_4$: disturbance derivative',      'C2')]
for a, (v, lb, c) in zip(axs.flat, est):
```

```
    a.step(p.index, p[v], c, lw=1.0); a.set_title(lb, fontsize=10); a.
 ↪set_xlabel('t [s]'); a.grid(alpha=0.35)
    a.set_xlim(0,1)
fig.suptitle('States estimated by the observer', fontsize=12, fontweight='bold')
plt.show()
```



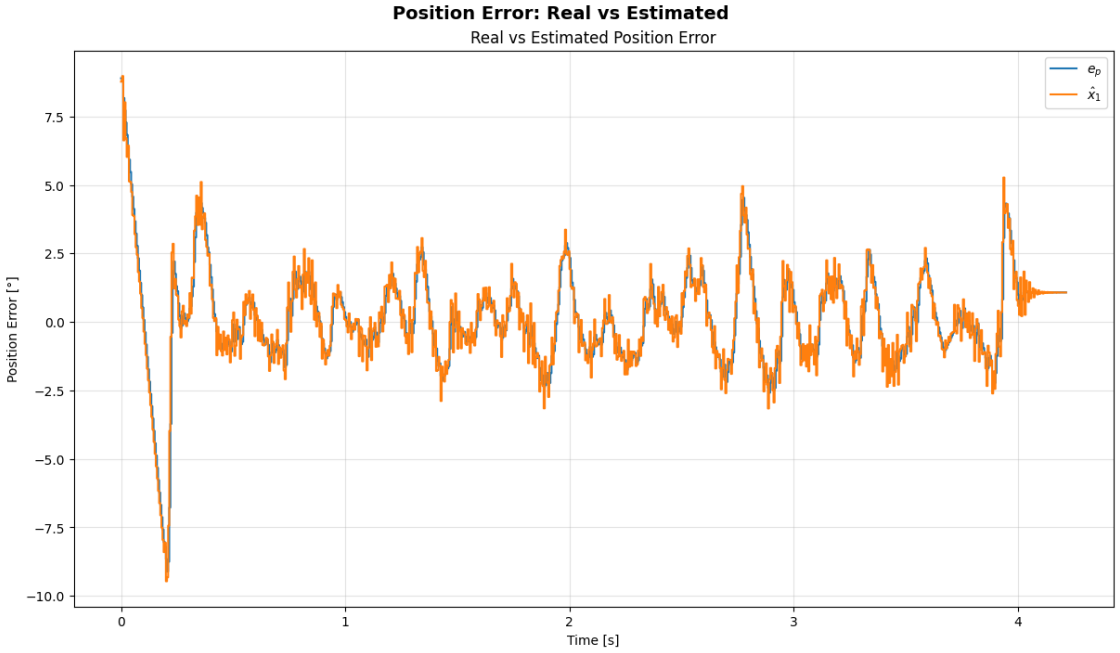
[20]:
```
fig, axs = plt.subplots(1, 1, figsize=(12, 7), layout='constrained')

# Plot
axs.step(p.index, p['ep'], lw=1.5, label=r'$e_p$')
axs.step(p.index, p['xhat0'], lw=1.5, label=r'$\hat{x}_1$')

# Labels and formatting
axs.set_title('Real vs Estimated Position Error', fontsize=12)
axs.set_xlabel('Time [s]')
axs.set_ylabel('Position Error [°]')
axs.grid(alpha=0.35)
axs.legend()

fig.suptitle('Position Error: Real vs Estimated', fontsize=14,␣
 ↪fontweight='bold')

plt.show()
```



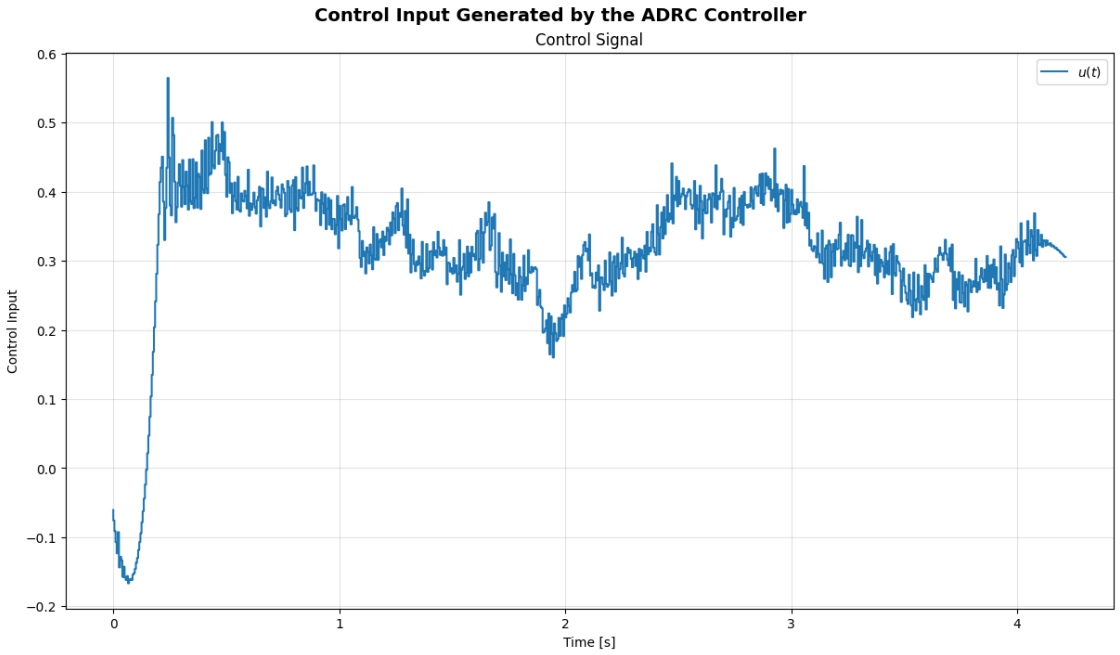
[25]:
```
fig, axs = plt.subplots(1, 1, figsize=(12, 7), layout='constrained')

# Control signal
axs.step(p.index, p['u'], lw=1.5, label=r'$u(t)$')

# Labels and formatting
axs.set_title('Control Signal', fontsize=12)
axs.set_xlabel('Time [s]')
axs.set_ylabel('Control Input')
axs.grid(alpha=0.35)
axs.legend()

fig.suptitle('Control Input Generated by the ADRC Controller',
             fontsize=14, fontweight='bold')

plt.show()
```

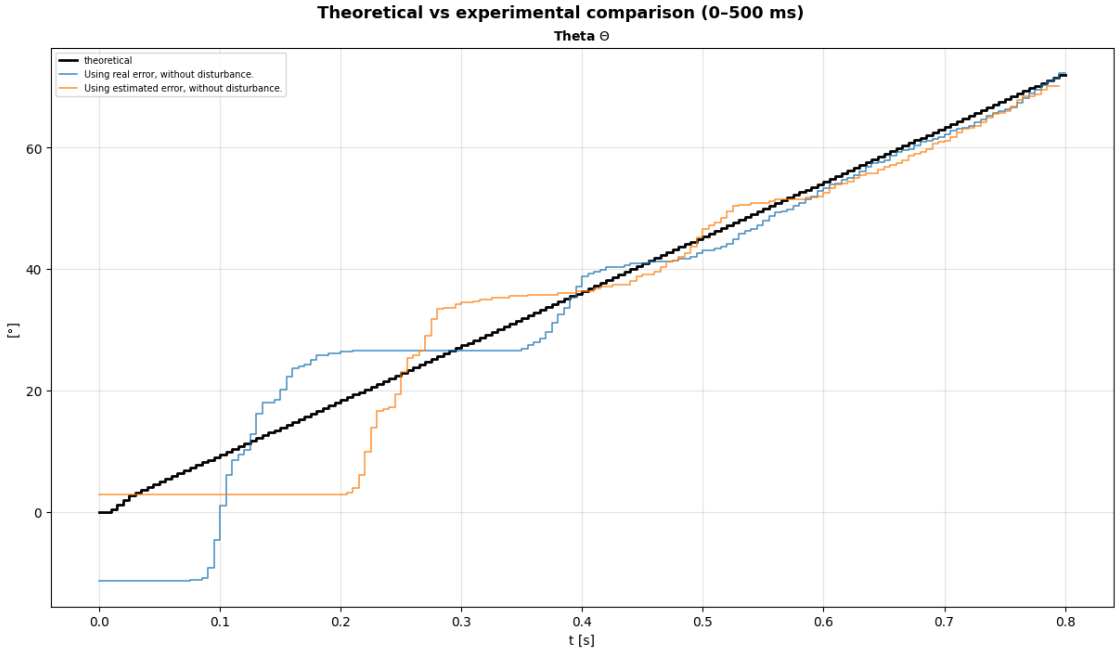


4.0.3 Comparación teórico vs experimental (0–500 ms)

[22]:
```
mt = TEO['t'] <= 0.8
fig, axs = plt.subplots(1, 1, figsize=(12,7), layout='constrained')
def _cmp(a, v, ttl, yl):
    a.step(TEO['t'][mt], TEO[v][mt], 'k', lw=2, label='theoretical')
    for k in ['real_sin','estim_sin']:
        pp = R[k][R[k].index <= 0.8]; a.step(pp.index, pp[v], lw=1.1, alpha=0.
 ↪85, label=LB[k])
    a.set_title(ttl, fontsize=10, fontweight='bold'); a.set_xlabel('t [s]'); a.
 ↪set_ylabel(yl)
    a.grid(alpha=0.35); a.legend(fontsize=7)
_cmp(axs, 'theta',    'Theta $\Theta$', '[°]')

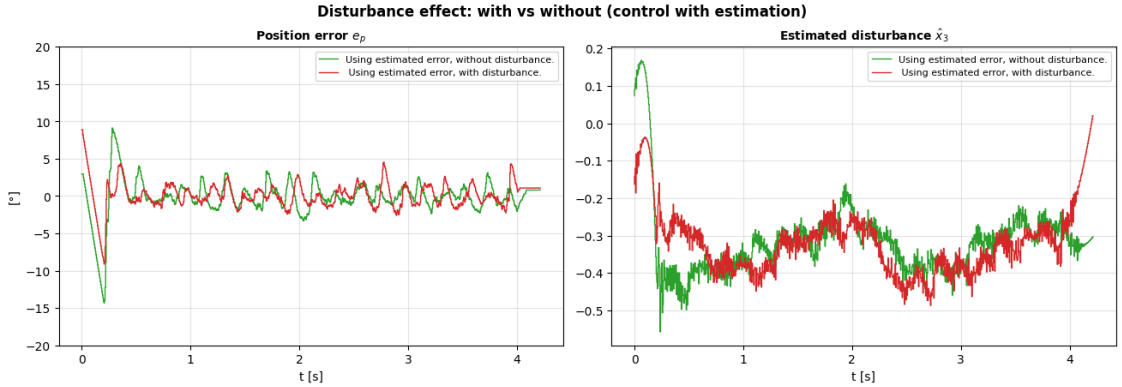
fig.suptitle('Theoretical vs experimental comparison (0-500 ms)', fontsize=13,␣
 ↪fontweight='bold')
plt.show()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\T'
<>:9: SyntaxWarning: invalid escape sequence '\T'
/tmp/ipykernel_7830/1750819858.py:9: SyntaxWarning: invalid escape sequence '\T'
  _cmp(axs, 'theta',    'Theta $\Theta$', '[°]')
```



4.0.4 Efecto de la perturbación (con vs sin) sobre el error y la perturbación estimada

[23]:
```
fig, axs = plt.subplots(1, 2, figsize=(13,4.5), layout='constrained')
for k, c in [('estim_sin','C2'), ('estim_con','C3')]:
    p = R[k]
    axs[0].step(p.index, p['ep'],    c, lw=1.0, label=LB[k])
    axs[1].step(p.index, p['xhat2'], c, lw=1.0, label=LB[k])
axs[0].set_title('Position error $e_p$', fontsize=10, fontweight='bold');␣
 ↪axs[0].set_xlabel('t [s]'); axs[0].set_ylabel('[°]'); axs[0].
 ↪set_ylim(-20,20); axs[0].legend(fontsize=8); axs[0].grid(alpha=0.35)
axs[1].set_title(r'Estimated disturbance $\hat{x}_3$', fontsize=10,␣
 ↪fontweight='bold'); axs[1].set_xlabel('t [s]'); axs[1].legend(fontsize=8);␣
 ↪axs[1].grid(alpha=0.35)
fig.suptitle('Disturbance effect: with vs without (control with estimation)',␣
 ↪fontsize=12, fontweight='bold')
plt.show()
```



4.0.5 Tabla resumen de desempeño (subida 0°→360°)

[24]:
```
filas = []
mt = TEO['t'] <= 4.0
filas.append(['Teórico', np.abs(TEO['ep'][mt]).mean(), np.abs(TEO['ep'][mt]).
 ↪max(), np.abs(TEO['u'][mt]).max()])
for k in R:
    p = R[k]; filas.append([LB[k], np.abs(p['ep']).mean(), np.abs(p['ep']).
 ↪max(), np.abs(p['u']).max()])
tabla = pd.DataFrame(filas, columns=['Caso','|e_p| medio [°]','|e_p| máx␣
 ↪[°]','|u| máx']).round(2)
display(tabla)
```

```
                                    Caso  |e_p| medio [°]  \
0                                Teórico             0.01
1          Using real error, with disturbance            1.41
2       Using real error, without disturbance.           1.78
3     Using estimated error, with disturbance.           1.26
4  Using estimated error, without disturbance.           1.50

   |e_p| máx [°]  |u| máx
0           0.92     0.05
1          10.55     0.48
2          17.53     0.51
3           9.11     0.49
4          14.32     0.56
```

$$\hat{x}(k+1) = (A - LC)\,\hat{x}(k) + Bu(k) + Le_y(k)$$

$$\hat{x}(k) = (A - LC)\,\hat{x}(k-1) + Bu(k-1) + Le_y(k-1)$$